Report : Chat

Made by MARFIL Elouann & GAUCHIER Alexandre

Github:

Lien Github

Software Architecture :

Overall Architecture

The application implements a distributed client-server model based on Java RMI (Remote Method Invocation). The architecture relies on three main components: the RMI Registry, which acts as a service directory; one or more chat servers that manage conversations; and clients that connect to these servers to exchange messages.

Main Components

- RMI Registry :

A directory service that allows clients to dynamically discover available servers on the network. Each server registers itself with a unique name, enabling multiple servers to coexist simultaneously within the same registry. Clients can list all available services and choose the server to connect to.

- Server :

Maintains the list of connected clients in a `ConcurrentHashMap`, distributes messages to all clients via RMI callbacks, and persists the message history in a text file that is reloaded at startup. A shutdown hook ensures a clean shutdown by notifying clients and unregistering from the registry.

- Client :

The client is structured according to an MVC architecture:

- **View (ClientVue):** Swing interface that displays messages
- **Controller (ChatClientController):** Connects the view to the RMI server

Communication Pattern

The architecture uses bidirectional communication:

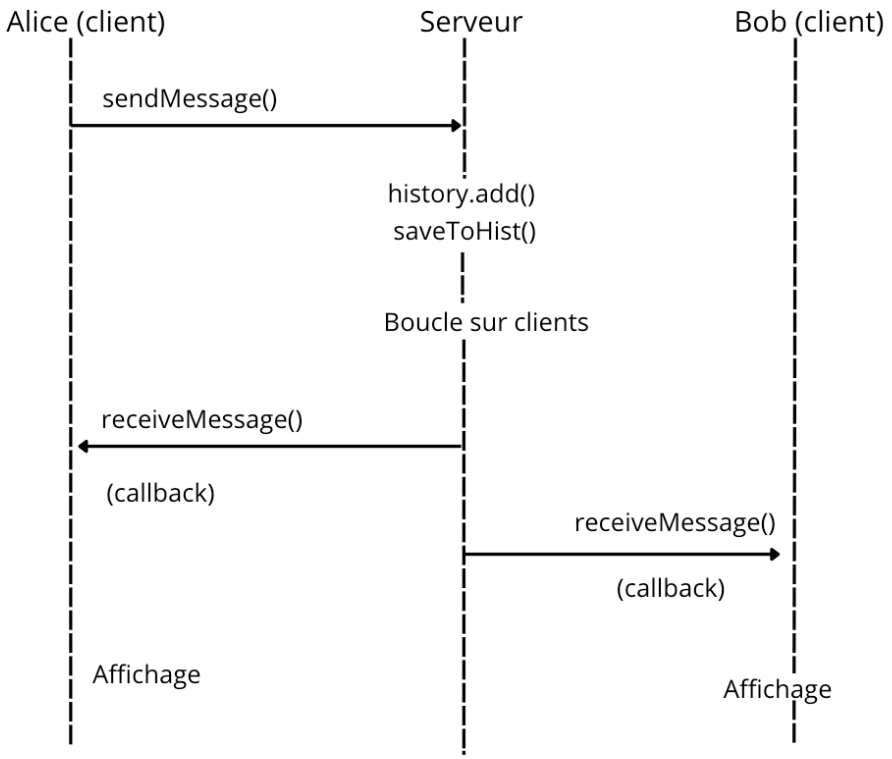
- **Client → Server:** Standard RMI calls (`sendMessage()`, `joinChat()`, etc.)
- **Server → Client:** RMI callbacks (`receiveMessage()`, `joinChat()`, `leaveChat()`, etc.)

When a client connects, it exports itself as a remote object and sends its reference to the server. The server stores this reference and can then invoke methods on the client.

When a user sends a message, the server immediately distributes it to all connected clients without requiring them to periodically poll the server.

Example of Interaction

1. Alice sends a message via `sendMessage()` (RMI call)
2. The server stores the message in the history (memory + file)
3. The server iterates over all connected clients
4. The server invokes `receiveMessage()` on each client (RMI callbacks)
5. Each client displays the message in its interface



Concurrency Management

Server Side

- **`ConcurrentHashMap`:** Stores connected clients in a thread-safe manner. Multiple threads can simultaneously add, remove, or iterate over clients without risking a `ConcurrentModificationException`.

- **`CopyOnWriteArrayList`:** Stores the message history in a thread-safe manner. This structure is optimized for frequent reads (e.g., the `/hist` command) and occasional writes (adding messages). Each modification creates a new copy of the internal array, allowing readers to iterate over the history without blocking, even while new messages are being added.

Client Side

All graphical interface updates are wrapped in calls to `SwingUtilities.invokeLater()`. This method delegates the execution of a task to the Event Dispatch Thread (EDT), the only thread allowed to modify Swing components. Without this protection, RMI callbacks running in separate threads would directly update the interface, causing race conditions and unpredictable behavior, since Swing is not thread-safe.

Fault Tolerance and Failure Handling

The server implements basic failure-handling mechanisms. Client crashes or abrupt disconnections are detected during RMI callbacks: if a callback invocation throws a RemoteException, the client is considered unreachable and removed from the connected clients map, while message broadcasting continues for the remaining users.

In addition, a graceful shutdown mechanism is implemented using a shutdown hook. Before the server terminates, the hook notifies all connected clients through a dedicated remote method. Upon receiving this notification, each client displays a short message to the user and then cleanly terminates. This ensures that clients do not remain in an inconsistent state when the server stops.

Application Features :

The chat application allows users to:

- Connect to different chat servers available on the network
- Send and receive real-time messages with all connected users
- Send private messages to users connected to the same server
- View the list of connected users
- Manage message history (viewing and clearing)

To make these features available to the client, the following commands are provided:

Command	Description
message	*Send a message to the chat*
/quit	*Leave the chat*
/hist	*Display the chat message history*
/users	*Display the list of connected users*
/clear	*Clear the chat message history*
/pm <user> <message>	*Send a private message*

Compilation and Execution Script :

Step	Step Title	With Makefile	Without Makefile
1	Start the RMI Registry	make rmi	cd bin && rmiregistry <port> &
2	Compile the sources	make	javac -d bin *.java
3	Start a server	make server	java -cp bin ChatServer <port> <serverName>

4	Start a client	make client	java -cp bin ChatClient localhost <port>

It is also possible to start a server with a custom name using the following command:

```
make server ARGS="ServerName"
```

Usage :

When launching the client, two dialog windows appear successively:

- Selection of the chat server to connect to
- Entry of the desired username

Once the client is connected and identified, it can interact with the server and the connected users using the available commands.